



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

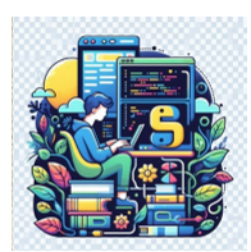
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Aggregation

Lekha A

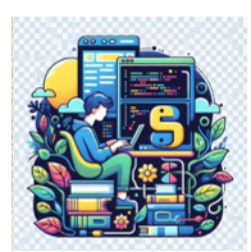
Computer Applications



PROGRAMMING WITH PYTHON

Recap

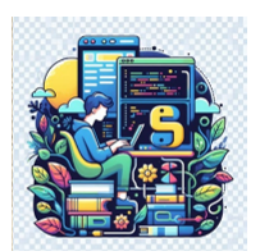
- Composition
- Examples
- Scenarios for Composition



PROGRAMMING WITH PYTHON

Aggregation

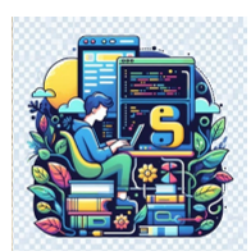
- Aggregation is almost exactly like composition.
- The difference is that aggregate objects can exist independently.
- Aggregation is a concept in which an object of one class can own or access another independent object of another class.
- It represents Has-A's relationship.



PROGRAMMING WITH PYTHON

Aggregation

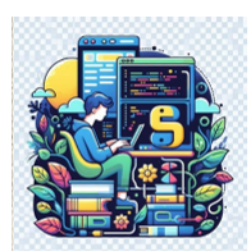
- It is a unidirectional association i.e. a one-way relationship.
 - For example, a department can have students but vice versa is not possible and thus unidirectional in nature.
- In Aggregation, both the entries can survive individually which means ending one entity will not affect the other entity.



PROGRAMMING WITH PYTHON

Aggregation

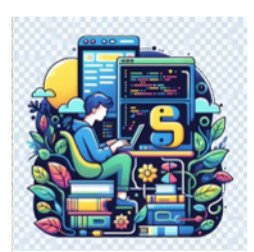
- In Python, aggregation refers to a type of object relationship where one object has a "has-a" relationship with another object.
- This means that the first object contains a reference to the second object, but the second object does not necessarily belong exclusively to the first object.
- Aggregation is a weaker form of association than composition, as the second object can exist independently of the first object.



PROGRAMMING WITH PYTHON

Aggregation

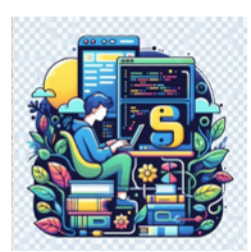
- Aggregation is a useful tool for creating object hierarchies and modeling real-world relationships.
- It allows creation of complex objects by combining simpler objects, and it promotes code modularity and reusability.



PROGRAMMING WITH PYTHON

Reasons for Aggregation

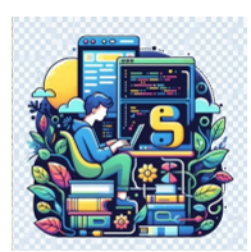
- Modeling Relationships
 - Aggregation is a natural way to model "has-a" relationships between objects.
 - For example, a university can have a department, a department can have professors, and a professor can teach courses.
 - These relationships can be modeled using aggregation.



PROGRAMMING WITH PYTHON

Reasons for Aggregation

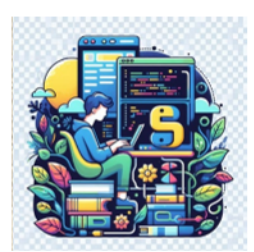
- Promoting Modularity
 - Aggregation encourages code modularity by allowing the break down of complex objects into smaller, more manageable components.
 - This makes the code easier to understand, maintain, and test.



PROGRAMMING WITH PYTHON

Reasons for Aggregation

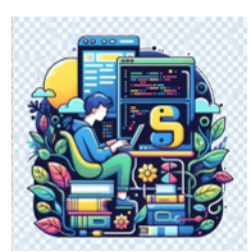
- Enabling Code Reuse
 - Aggregated objects can be reused in different parts of the application, reducing code duplication and promoting consistency.
 - This reusability saves time and effort, making the code more efficient.



PROGRAMMING WITH PYTHON

Reasons for Aggregation

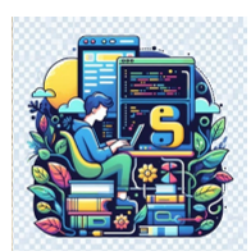
- Encapsulation and Information Hiding
 - Aggregation supports encapsulation by allowing the hiding of the implementation details of aggregated objects from the user of the parent object.
 - This promotes information hiding and reduces the risk of introducing bugs.



PROGRAMMING WITH PYTHON

Reasons for Aggregation

- Flexibility and Adaptability
 - Aggregation provides flexibility in designing object structures and adapting to changing requirements.
 - Aggregated objects can be easily added or deleted without affecting the overall structure of the parent object.

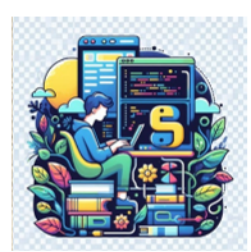


PROGRAMMING WITH PYTHON

Aggregation

```
class Teacher:
    def __init__(self, name, subject):
        self.name = name
        self.subject = subject

    def teach(self):
        print(f"{self.name} is teaching {self.subject}.")
```



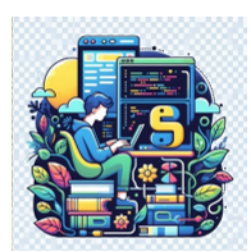
PROGRAMMING WITH PYTHON

Aggregation

```
class School:
    def __init__(self, school_name):
        self.school_name = school_name
        self.teachers = [] # List to hold Teacher objects

    def add_teacher(self, teacher):
        self.teachers.append(teacher)
        print(f"{teacher.name} has joined {self.school_name}.")

    def show_teachers(self):
        print(f"\nTeachers at {self.school_name}:")
        for t in self.teachers:
            print(f"- {t.name} ({t.subject})")
```



PROGRAMMING WITH PYTHON

Aggregation

```
t1 = Teacher("Filius Flitwick", "Charms")  
t2 = Teacher("Minerva McGonagall", "Transfiguration")
```

```
school = School("Hogwarts School of Witchcraft and Wizardry")
```

```
school.add_teacher(t1)  
school.add_teacher(t2)
```

Filius Flitwick has joined Hogwarts School of Witchcraft and Wizardry.
Minerva McGonagall has joined Hogwarts School of Witchcraft and Wizardry.

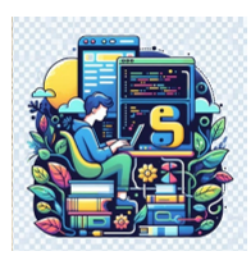
```
t1.teach()  
t2.teach()
```

Filius Flitwick is teaching Charms.
Minerva McGonagall is teaching Transfiguration.

```
school.show_teachers()
```

Teachers at Hogwarts School of Witchcraft and Wizardry:

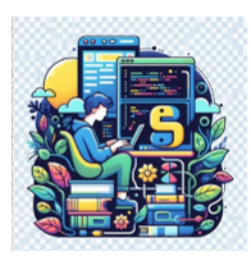
- Filius Flitwick (Charms)
- Minerva McGonagall (Transfiguration)



PROGRAMMING WITH PYTHON

Aggregation

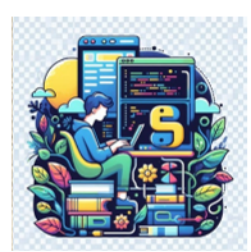
- This code demonstrates aggregation by creating objects of different classes and connecting them through references.
- The School class aggregates Teacher objects.



PROGRAMMING WITH PYTHON

Aggregation

- These relationships represent "has-a" relationships, where the parent object owns a reference to the child object, but the child object can exist independently.

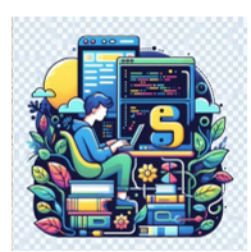


PROGRAMMING WITH PYTHON

Aggregation

```
class Character:  
    def __init__(self, char):  
        self.char = char  
  
    def __str__(self):  
        return self.char
```

```
class Word:  
    def __init__(self):  
        self.characters = []  
  
    def add_character(self, char):  
        self.characters.append(Character(char))  
  
    def __str__(self):  
        return ''.join(str(char) for char in self.characters)
```



PROGRAMMING WITH PYTHON

Aggregation

```
class Paragraph:
    def __init__(self):
        self.words = []

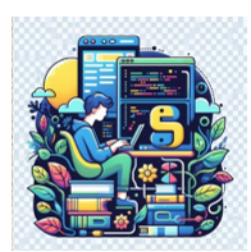
    def add_word(self, word):
        self.words.append(word)

    def __str__(self):
        return ' '.join(str(word) for word in self.words)
```

```
class File:
    def __init__(self):
        self.paragraphs = []

    def add_paragraph(self, paragraph):
        self.paragraphs.append(paragraph)

    def __str__(self):
        return '\n'.join(str(paragraph) for paragraph in self.paragraphs)
```



PROGRAMMING WITH PYTHON

Aggregation

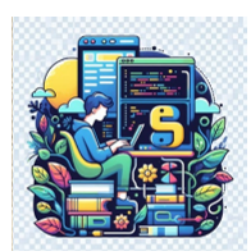
```
class Folder:
    def __init__(self):
        self.files = []

    def add_file(self, file):
        self.files.append(file)

    def __str__(self):
        return '\n\n'.join(str(file) for file in self.files)
```

```
class User:
    def __init__(self):
        self.folders = [Folder(), Folder()]

    def __str__(self):
        return '\n\n\n'.join(str(folder) for folder in self.folders)
```



PROGRAMMING WITH PYTHON

Aggregation

- `user = User()`

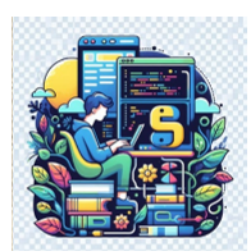
```
# First folder
file1 = File()
paragraph1 = Paragraph()

word1 = Word()
word1.add_character('H')
word1.add_character('e')
word1.add_character('l')
word1.add_character('l')
word1.add_character('o')
```

```
word2 = Word()
word2.add_character('W')
word2.add_character('o')
word2.add_character('r')
word2.add_character('l')
word2.add_character('d')

paragraph1.add_word(word1)
paragraph1.add_word(word2)
file1.add_paragraph(paragraph1)

user.folders[0].add_file(file1)
```



PROGRAMMING WITH PYTHON

Aggregation

- *# Second folder*

```
file2 = File()
```

```
paragraph2 = Paragraph()
```

```
word3 = Word()
```

```
word3.add_character('P')
```

```
word3.add_character('y')
```

```
word3.add_character('t')
```

```
word3.add_character('h')
```

```
word3.add_character('o')
```

```
word4 = Word()
```

```
word4.add_character('n')
```

```
word4.add_character('!')
```

```
word4.add_character('!')
```

```
word4.add_character('!')
```

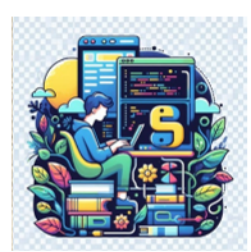
```
word4.add_character('!')
```

```
paragraph2.add_word(word3)
```

```
paragraph2.add_word(word4)
```

```
file2.add_paragraph(paragraph2)
```

```
user.folders[1].add_file(file2)
```



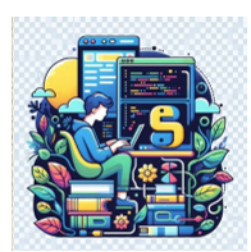
PROGRAMMING WITH PYTHON

Aggregation

- ```
print(user)
```

Hello World

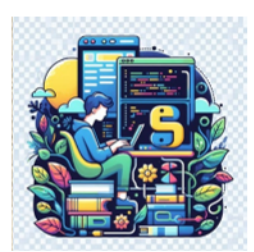
Pytho n!!!!



# PROGRAMMING WITH PYTHON

## Aggregation

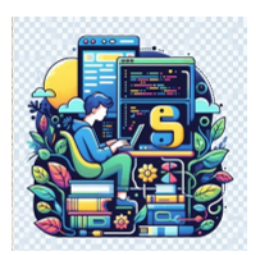
- This code represents a structure where a User has a Folder, which contains Files.
- Each File contains Paragraphs, each Paragraph contains Words, and each Word contains Characters.
  - This implements both aggregation and composition.



# PROGRAMMING WITH PYTHON

## Aggregation

| Feature                  | Composition                                    | Aggregation                                   |
|--------------------------|------------------------------------------------|-----------------------------------------------|
| Relationship             | Part-of                                        | Has-a                                         |
| Child object's existence | Dependent on parent object                     | Independent of parent object                  |
| Implementation           | Child object is contained within parent object | Parent object has a reference to child object |



# PROGRAMMING WITH PYTHON

## Recap

- Aggregation
- Reasons for aggregation
- Examples
- Aggregation vs. Composition



**THANK YOU**

---

**Lekha A**  
Computer Applications